VULNERABILITY TESTING / LAB REPORT

OS COMMAND INJECTION

Web Application Penetration Testing – Internship Report

Organization: IqraSec Academy
Platform: TryHackMe

COVER PAGE

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT)
Topic	OS Command Injection
Platform	TryHackMe
Number of Labs	1
Completion Date	25 October 2026

This report documents the objective, vulnerability concept, testing methodology, evidence, impact, remediation, and learning outcomes of the OS Command Injection lab.

Lab 1: OS Command Injection

1. Lab Name

OS Command Injection

Platform: TryHackMe
Room URL: https://tryhackme.com/room/oscommandinjection

2. Objective

The objective of this lab is to understand Operating System (OS) Command Injection vulnerabilities in web applications and learn how insecure handling of user input can allow unintended operating-system commands to be executed.

The main objectives are:

- Understand the concept of OS Command Injection.
- Identify application functionality that may interact with system commands.
- Understand how unsanitized input can affect command execution.
- Learn the difference between in-band and blind command injection.
- Understand the security risks of unsafe command construction.
- Learn secure coding practices to prevent command injection.

3. Vulnerability / Concept

What is OS Command Injection?

OS Command Injection is a security vulnerability that occurs when an application constructs an operating-system command using untrusted input without adequate safeguards.

If an application passes that input to a system shell unsafely, an attacker may be able to alter the intended command or cause additional commands to execute.

The potential impact depends on the privileges of the application process and the commands that can be executed.

Types of Command Injection

1. In-Band Command Injection

The results of command execution are returned through the application's normal response or displayed directly to the user.

2. Blind Command Injection

Commands may execute without their output appearing in the application's response. Their execution may instead be inferred from observable behavior, such as a response delay or another controlled side effect.

3. Error-Based Indicators

Errors or unexpected application responses may reveal that user input is reaching a command-processing component. An error alone does not prove that command injection is exploitable.

Common Vulnerable Functionality

- Network diagnostic tools.
- Ping or traceroute features.
- File-processing operations.
- Administrative utilities.
- Applications that invoke shell commands using user-supplied parameters.

Common Indicators

- Unexpected command-related output appears in an application response.
- Input changes the behavior of a system utility.

- Application responses show unusual delays or errors after controlled testing.
- The application constructs shell commands by concatenating user input.

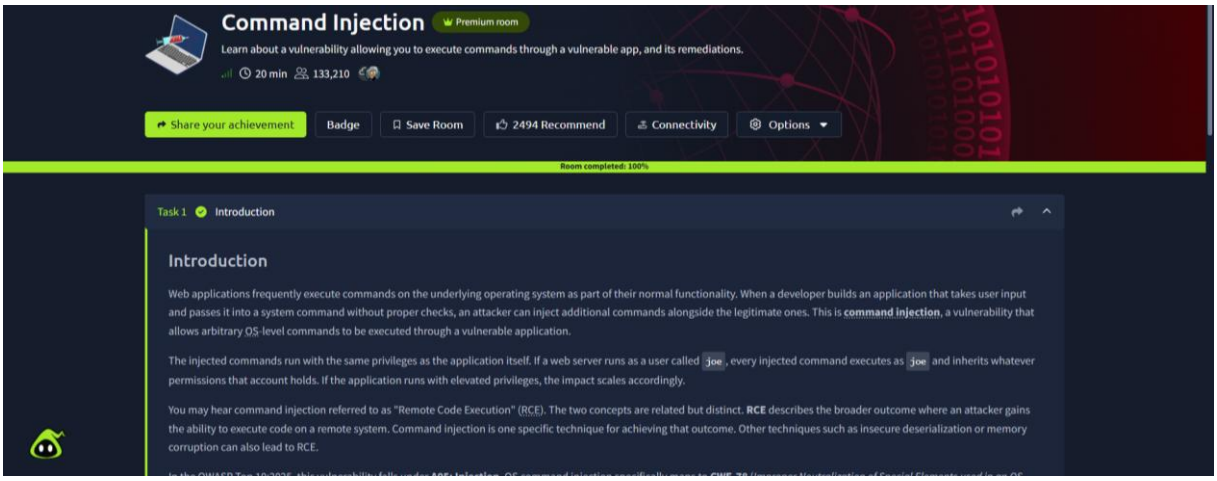
4. Testing Approach

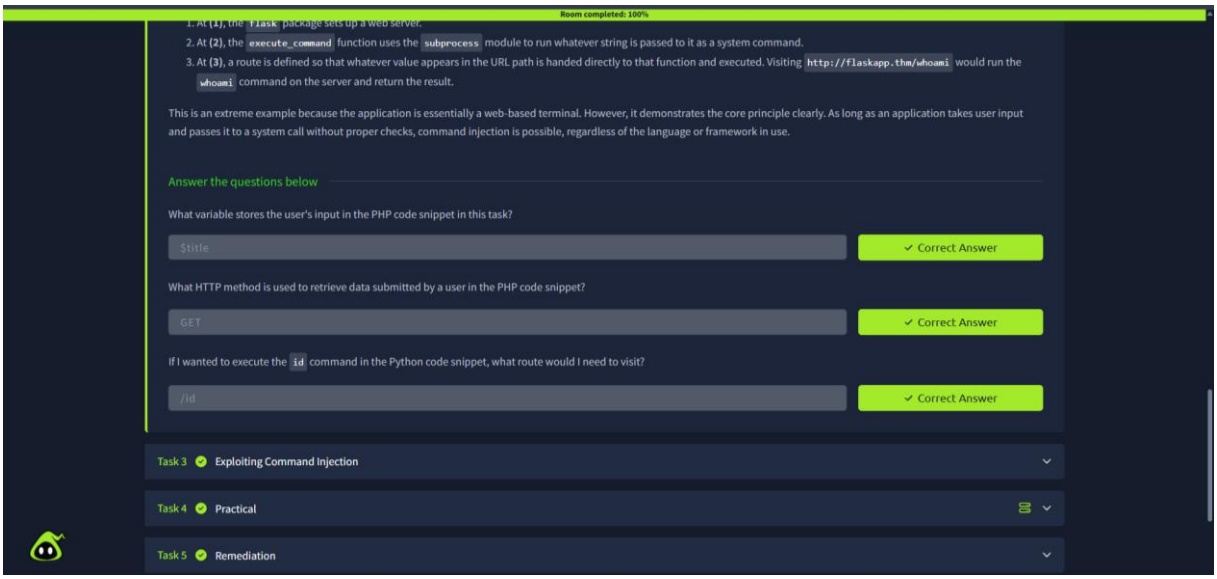
The following workflow can be used within the authorized TryHackMe lab environment.

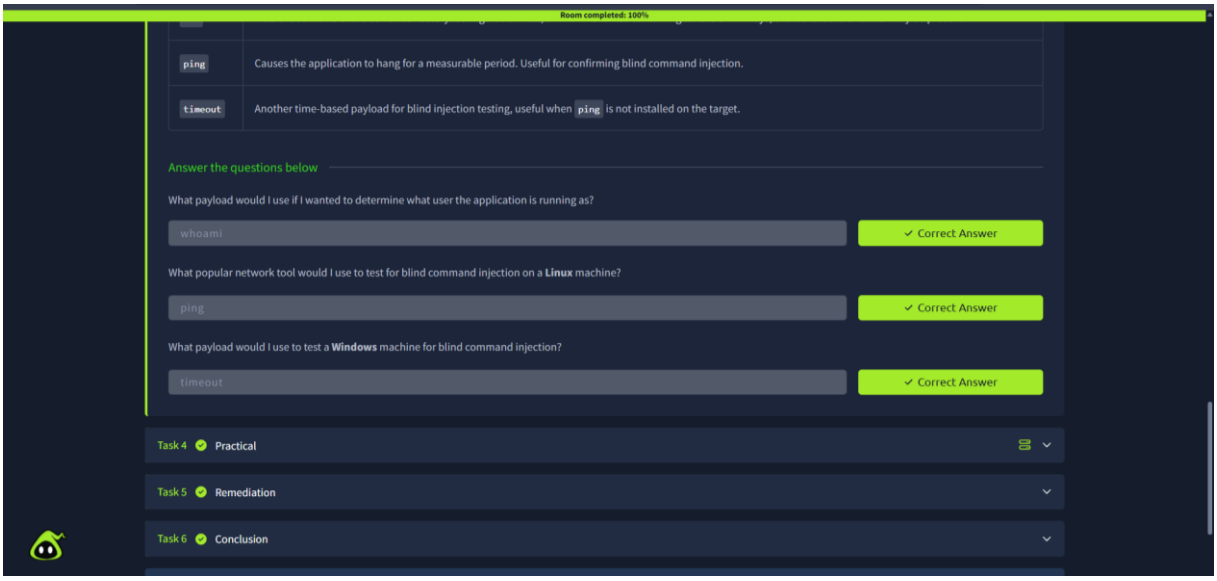
1. Open the OS Command Injection room on TryHackMe.
2. Read the room instructions and learning objectives.
3. Start the provided lab environment if required.
4. Open the target web application in the provided browser.
5. Identify features that may invoke operating-system utilities.
6. Establish normal application behavior using valid lab-provided input.
7. Observe the parameters and requests associated with the selected feature.
8. Review how the application processes the supplied input.
9. Follow the room's instructions to examine command injection concepts.
10. Perform the designated test using the lab's provided methodology.
11. Observe the application response and record any unexpected output.
12. Determine whether the response indicates normal behavior, an error, or possible command execution.
13. Review in-band command injection and how output may be returned.
14. Review blind command injection and the observable behavior used in the lab.
15. Compare the observed behavior with the application's expected functionality.
16. Record the relevant request, response, and testing conditions.
17. Capture screenshots that demonstrate the actual test and result.
18. Identify the potential impact based on the behavior demonstrated.
19. Recommend appropriate secure coding and input-handling controls.
20. Document the confirmed findings and complete the lab's assigned tasks.

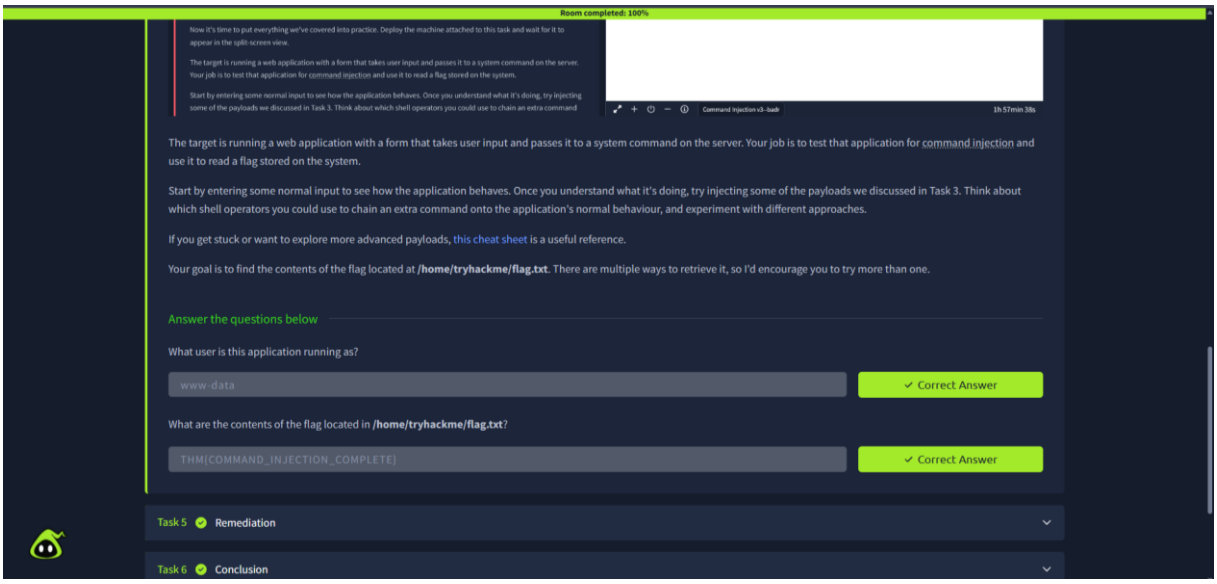
Testing note: Perform testing only against the authorized TryHackMe environment. Record only tests actually performed and results actually observed. Do not claim successful command execution unless the evidence supports it.

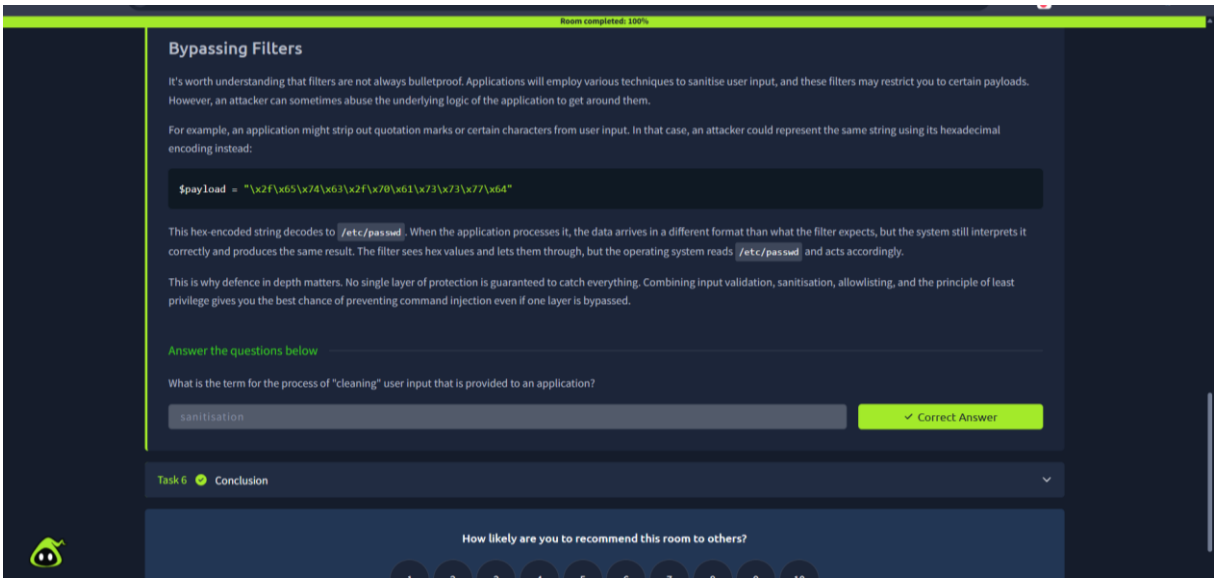
5. Evidence / Screenshots

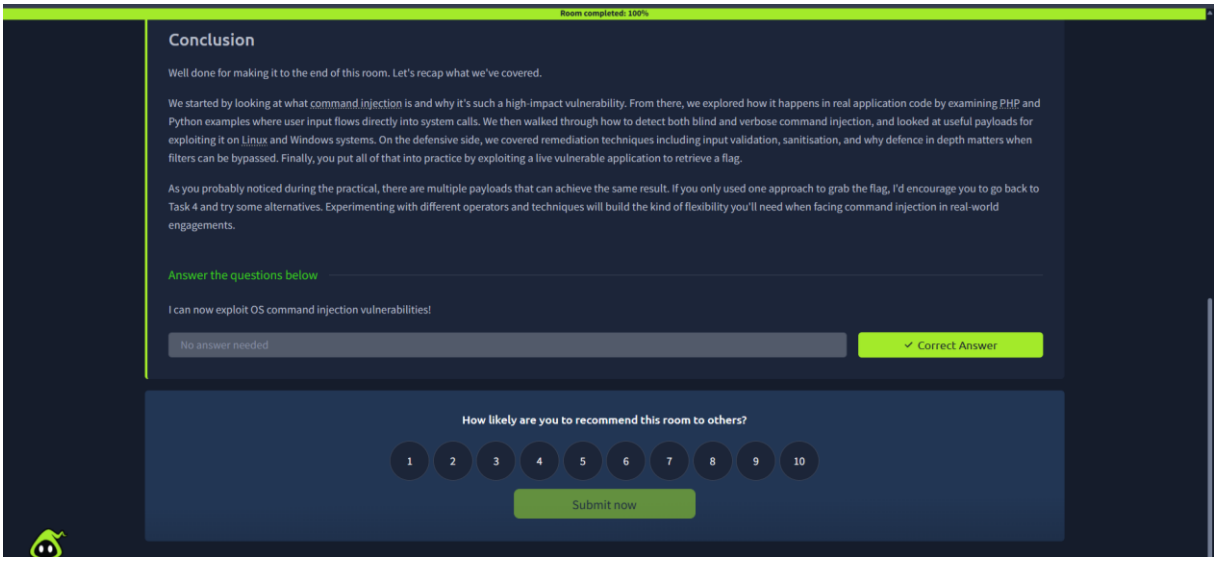

Command Injection
Premium room
Learn about a vulnerability allowing you to execute commands through a vulnerable app, and its remediations.
20 min
133,210
Share your achievement
Badge
Save Room
2494 Recommend
Connectivity
Options
Room completed: 100%
Task 1 Introduction
Introduction
Web applications frequently execute commands on the underlying operating system as part of their normal functionality. When a developer builds an application that takes user input and passes it into a system command without proper checks, an attacker can inject additional commands alongside the legitimate ones. This is command injection, a vulnerability that allows arbitrary OS-level commands to be executed through a vulnerable application.
The injected commands run with the same privileges as the application itself. If a web server runs as a user called joe, every injected command executes as joe and inherits whatever permissions that account holds. If the application runs with elevated privileges, the impact scales accordingly.
You may hear command injection referred to as "Remote Code Execution" (RCE). The two concepts are related but distinct. RCE describes the broader outcome where an attacker gains the ability to execute code on a remote system. Command injection is one specific technique for achieving that outcome. Other techniques such as insecure deserialization or memory corruption can also lead to RCE.


Room completed: 100%
1. At (1), the flask package sets up a web server.
2. At (2), the execute_command function uses the subprocess module to run whatever string is passed to it as a system command.
3. At (3), a route is defined so that whatever value appears in the URL path is handed directly to that function and executed. Visiting http://flaskapp.thm/whoami would run the whoami command on the server and return the result.
This is an extreme example because the application is essentially a web-based terminal. However, it demonstrates the core principle clearly. As long as an application takes user input and passes it to a system call without proper checks, command injection is possible, regardless of the language or framework in use.
Answer the questions below
What variable stores the user's input in the PHP code snippet in this task?
$title
✓ Correct Answer
What HTTP method is used to retrieve data submitted by a user in the PHP code snippet?
GET
✓ Correct Answer
If I wanted to execute the id command in the Python code snippet, what route would I need to visit?
/id
✓ Correct Answer
Task 3 Exploiting Command Injection
Task 4 Practical
Task 5 Remediation


Room completed: 100%
ping
Causes the application to hang for a measurable period. Useful for confirming blind command injection.
timeout
Another time-based payload for blind injection testing, useful when ping is not installed on the target.
Answer the questions below
What payload would I use if I wanted to determine what user the application is running as?
whoami
✓ Correct Answer
What popular network tool would I use to test for blind command injection on a Linux machine?
ping
✓ Correct Answer
What payload would I use to test a Windows machine for blind command injection?
timeout
✓ Correct Answer
Task 4 Practical
Task 5 Remediation
Task 6 Conclusion


Room completed: 100%
Now it's time to put everything we've covered into practice. Deploy the machine attached to this task and wait for it to appear in the split-screen view.
The target is running a web application with a form that takes user input and passes it to a system command on the server. Your job is to test that application for command injection and use it to read a flag stored on the system.
Start by entering some normal input to see how the application behaves. Once you understand what it's doing, try injecting some of the payloads we discussed in Task 3. Think about which shell operators you could use to chain an extra command
Command Injection v3 - bwdb
1h 57min 38s
The target is running a web application with a form that takes user input and passes it to a system command on the server. Your job is to test that application for command injection and use it to read a flag stored on the system.
Start by entering some normal input to see how the application behaves. Once you understand what it's doing, try injecting some of the payloads we discussed in Task 3. Think about which shell operators you could use to chain an extra command onto the application's normal behaviour, and experiment with different approaches.
If you get stuck or want to explore more advanced payloads, this cheat sheet is a useful reference.
Your goal is to find the contents of the flag located at /home/tryhackme/flag.txt. There are multiple ways to retrieve it, so I'd encourage you to try more than one.
Answer the questions below
What user is this application running as?
www-data
✓ Correct Answer
What are the contents of the flag located in /home/tryhackme/flag.txt?
THM{COMMAND_INJECTION_COMPLETE}
✓ Correct Answer
Task 5 Remediation
Task 6 Conclusion


Room completed: 100%
Bypassing Filters
It's worth understanding that filters are not always bulletproof. Applications will employ various techniques to sanitise user input, and these filters may restrict you to certain payloads. However, an attacker can sometimes abuse the underlying logic of the application to get around them.
For example, an application might strip out quotation marks or certain characters from user input. In that case, an attacker could represent the same string using its hexadecimal encoding instead:
$payload = "\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64"
This hex-encoded string decodes to /etc/passwd. When the application processes it, the data arrives in a different format than what the filter expects, but the system still interprets it correctly and produces the same result. The filter sees hex values and lets them through, but the operating system reads /etc/passwd and acts accordingly.
This is why defence in depth matters. No single layer of protection is guaranteed to catch everything. Combining input validation, sanitisation, allowlisting, and the principle of least privilege gives you the best chance of preventing command injection even if one layer is bypassed.
Answer the questions below
What is the term for the process of "cleaning" user input that is provided to an application?
sanitisation
✓ Correct Answer
Task 6 Conclusion
How likely are you to recommend this room to others?


Room completed: 100%
Conclusion
Well done for making it to the end of this room. Let's recap what we've covered.
We started by looking at what command injection is and why it's such a high-impact vulnerability. From there, we explored how it happens in real application code by examining PHP and Python examples where user input flows directly into system calls. We then walked through how to detect both blind and verbose command injection, and looked at useful payloads for exploiting it on Linux and Windows systems. On the defensive side, we covered remediation techniques including input validation, sanitisation, and why defence in depth matters when filters can be bypassed. Finally, you put all of that into practice by exploiting a live vulnerable application to retrieve a flag.
As you probably noticed during the practical, there are multiple payloads that can achieve the same result. If you only used one approach to grab the flag, I'd encourage you to go back to Task 4 and try some alternatives. Experimenting with different operators and techniques will build the kind of flexibility you'll need when facing command injection in real-world engagements.
Answer the questions below
I can now exploit OS command injection vulnerabilities!
No answer needed
✓ Correct Answer
How likely are you to recommend this room to others?
1 2 3 4 5 6 7 8 9 10
Submit now

6. Impact

If successfully exploited, OS Command Injection may lead to the following consequences:

- **Unauthorized command execution:** An attacker may execute commands through the vulnerable application.
- **Sensitive information disclosure:** Commands may expose information accessible to the application's operating-system account.
- **Application compromise:** An attacker may interfere with application processes or files.
- **Privilege-dependent impact:** The consequences may be more severe if the application runs with excessive privileges.
- **Service disruption:** Commands may consume resources or disrupt application availability.
- **Further compromise:** In some circumstances, command execution may provide a route to additional system-level attacks.

The actual severity depends on the execution context, available system permissions, application configuration, and the behavior confirmed during testing.

7. Remediation

The following measures help prevent OS Command Injection vulnerabilities.

1. Avoid shell command construction

Prefer language-native libraries and APIs instead of invoking an operating-system shell.

2. Use parameterized process execution

Where external processes are necessary, use APIs that pass executable names and arguments separately rather than concatenating input into a shell command.

3. Validate input using allowlists

Accept only expected input formats, lengths, and values. Validation should reflect the application's actual requirements.

4. Avoid relying solely on character filtering

Removing selected shell characters is not a reliable primary defense. Use safer APIs and strict input validation.

5. Apply least privilege

Run application processes with only the operating-system permissions they need.

6. Restrict executable functionality

Limit which system utilities and operations the application can invoke.

7. Handle errors securely

Do not expose unnecessary system details, command output, or sensitive internal information in user-facing error messages.

8. Separate data from commands

Treat all user input as data, not executable instructions.

9. Log suspicious behavior

Record relevant security events and monitor repeated invalid inputs or unusual command-related behavior.

10. Perform security testing

Include command injection checks in code reviews, penetration tests, and regression testing.

8. What You Learned

The main learning outcomes of this lab are:

- Understanding the definition of OS Command Injection.
- Recognizing application features that may invoke system commands.
- Understanding the risks of passing untrusted input to a shell.
- Distinguishing in-band and blind command injection.
- Learning to examine requests and application responses during testing.
- Understanding the relationship between process privileges and exploitation impact.
- Learning secure alternatives to shell command construction.
- Understanding the importance of input validation and least privilege.
- Documenting security tests with evidence, findings, and remediation recommendations.

Update these learning outcomes according to the tasks and techniques you actually completed in the room.

Conclusion

OS Command Injection is a serious web application security risk when untrusted input is incorporated into operating-system commands without adequate protection. Depending on the application's privileges and configuration, successful exploitation may result in unauthorized command execution, information disclosure, or service disruption.

Secure development should prioritize avoiding shell invocation, using safe process-execution APIs, validating input, and enforcing least privilege.

This report provides a structured approach to studying command injection in an authorized lab environment. The final assessment should reflect the actual test results, supporting screenshots, and recorded completion status.

Report Prepared By: Rohit Kushwah
Organization: IqraSec Academy
Internship: Web Application Penetration Testing (WAPT)